MAT61-1

# Optimization

From Linear Programs to Gradient Descent and Beyond

---

Convexity is the line between problems we can solve and problems we can only hope to.

Albert School — 30 hours

# Contents



This course assumes calculus (derivatives, gradients, Taylor approximation, convexity via the second derivative) from MAT11-1 and linear algebra from MAT11-3, MAT21-1, and MAT31-1 — in particular matrices, eigenvalues, symmetric matrices, and the classification of a quadratic form as positive definite, semi-definite, or indefinite by the signs of its eigenvalues.

# 1 Linear programming

A **linear program** (LP) is an optimization problem in which the objective and all constraints are linear in the decision variables.

## 1.1 Formulation

To formulate a resource-allocation problem as an LP, identify three components:

- **Decision variables**: the quantities to be chosen, written $x_1, ..., x_n$.
- **Objective function**: a linear function $c^T x = \sum_{j=1}^{n} c_j x_j$ to be maximized or minimized.
- **Constraints**: linear inequalities or equalities the variables must satisfy, together with sign restrictions (typically $x_j \geq 0$).

## 1.2 Standard form

An LP is in **standard form** when it is written as

$$\max \quad c^T x \quad \text{subject to} \quad Ax \le b, \quad x \ge 0,$$

with $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, and $b \in \mathbb{R}^m$. Inequality constraints are converted to equalities by adding nonnegative **slack variables**: $a^T x \le \beta$ becomes $a^T x + s = \beta$ with $s \ge 0$. A minimization is turned into a maximization by negating the objective; a free variable is split as the difference of two nonnegative variables.

The set of points satisfying all constraints is the **feasible region**, a convex polyhedron. An optimal value, when it exists and is finite, is attained at a **vertex** (extreme point) of this polyhedron.

## 1.3 The simplex method

The **simplex method** moves from vertex to adjacent vertex of the feasible region, improving the objective at each step, until no adjacent vertex improves it.

Working from the equality form $Ax = b$, $x \ge 0$ with $m$ constraints and $n$ variables (including slacks), a **basic feasible solution** is obtained by choosing $m$ **basic variables**, setting the remaining $n - m$ **nonbasic variables** to 0, and solving for the basic ones; the solution is feasible if all basic variables are $\ge 0$. Each basic feasible solution corresponds to a vertex.

One iteration (pivot):

1. Express the objective in terms of the nonbasic variables. The coefficients there are the **reduced costs**.
2. If all reduced costs are $\le 0$ (for a maximization), the current vertex is optimal — stop. Otherwise pick a nonbasic variable with positive reduced cost to **enter** the basis.
3. **Ratio test**: increase the entering variable until some basic variable hits 0; that variable **leaves** the basis.
4. Pivot to update the basis and repeat.

## 1.4 Duality and shadow prices

Every LP (the **primal**) has an associated **dual** LP. For the primal

$$\max \quad c^T x \quad \text{s.t.} \quad Ax \le b, \quad x \ge 0,$$

the dual is

$$\min \quad b^T y \quad \text{s.t.} \quad A^T y \ge c, \quad y \ge 0.$$

There is one dual variable $y_i$ per primal constraint. **Weak duality**: for any primal-feasible $x$ and dual-feasible $y$, $c^T x \le b^T y$. **Strong duality**: if the primal has a finite optimum, so does the dual, and the optimal values are equal, $c^T x^* = b^T y^*$.

The optimal dual variable $y_i^*$ is the **shadow price** of constraint $i$: the rate of change of the optimal objective value per unit increase of the right-hand side $b_i$, valid for small changes. Economically, it is the marginal value of one more unit of resource $i$.

## 1.5 Complementary slackness

At a primal-dual optimal pair $(x^*, y^*)$, for every constraint and variable:

$$y_i^* \left( b_i - (Ax^*)_i \right) = 0 \quad \text{and} \quad x_j^* \left( (A^T y^*)_j - c_j \right) = 0.$$

That is, if a primal constraint is slack (holds with strict inequality) its dual variable is 0; and if a dual constraint is slack the corresponding primal variable is 0. Equivalently, a resource with spare capacity has shadow price 0. Complementary slackness, primal feasibility, and dual feasibility together characterize optimality, and can be used to verify a candidate solution.

# 2 Convexity

## 2.1 Convex sets and functions

A set $C \subseteq \mathbb{R}^n$ is **convex** if for all $x, y \in C$ and $\lambda \in [0,1]$, $\lambda x + (1-\lambda)y \in C$: the segment joining any two points of $C$ stays in $C$.

A function $f : C \to \mathbb{R}$ on a convex set $C$ is **convex** if

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda) f(y)$$

for all $x, y \in C$ and $\lambda \in [0,1]$ — the graph lies below every chord. It is **strictly convex** if the inequality is strict for $x \neq y$ and $\lambda \in (0,1)$. It is **concave** if $-f$ is convex.

## 2.2 Second-order condition

If $f$ is twice differentiable, $f$ is convex on an open convex set if and only if its **Hessian** $\nabla^2 f(x)$ is positive semi-definite (PSD) at every point:

$$\nabla^2 f(x) \succeq 0 \quad \text{for all } x.$$

If the Hessian is positive definite everywhere ($\succ 0$), $f$ is strictly convex (the converse need not hold). Positive (semi-)definiteness is read off the signs of the eigenvalues of the symmetric matrix $\nabla^2 f(x)$, as in MAT31-1.

## 2.3 Convexity-preserving operations

Convex functions can be combined into more complex convex objectives. If $f$ and $g$ are convex, then so are:

- **Nonnegative weighted sums**: $\alpha f + \beta g$ for $\alpha, \beta \geq 0$.
- **Pointwise maximum**: $\max(f, g)$, and the supremum of any family of convex functions.
- **Affine precomposition**: $x \mapsto f(Ax + b)$.
- **Composition** $h \circ f$ when $h$ is convex and nondecreasing and $f$ is convex.

These rules let one certify convexity of a composite objective without computing its Hessian directly.

## 2.4 Strict convexity and uniqueness

**Theorem 1 (Uniqueness of the minimizer)** A strictly convex function $f$ over a convex set $C$ has at most one global minimizer.

*Proof.* Suppose $x \neq y$ are both global minimizers, so $f(x) = f(y) = m$. By convexity of $C$ the midpoint $z = \frac{1}{2}x + \frac{1}{2}y$ lies in $C$, and by strict convexity

$$f(z) < \frac{1}{2} f(x) + \frac{1}{2} f(y) = m,$$

contradicting that $m$ is the minimum value. Hence there is at most one minimizer. □

For a convex function every **local** minimum is also **global**. This is why convexity is the dividing line between easy and hard optimization: on a convex problem any descent method that gets stuck has found the global optimum, whereas on a non-convex problem a method may halt at a local minimum or saddle point that is far from optimal (Section 8).

### 2.5 KKT conditions

Consider the constrained problem

$$\min \quad f(x) \quad \text{s.t.} \quad g_i(x) \le 0, \quad h_j(x) = 0.$$

The **Lagrangian** is $\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \nu_j h_j(x)$. The **Karush–Kuhn–Tucker (KKT)** first-order necessary conditions for $x^*$ to be optimal (under a constraint qualification) are:

- **Stationarity**: $\nabla f(x^*) + \sum_i \lambda_i \nabla g_i(x^*) + \sum_j \nu_j \nabla h_j(x^*) = 0$.
- **Primal feasibility**: $g_i(x^*) \le 0$ and $h_j(x^*) = 0$.
- **Dual feasibility**: $\lambda_i \ge 0$.
- **Complementary slackness**: $\lambda_i g_i(x^*) = 0$ for all $i$.

For a convex problem the KKT conditions are also **sufficient** for global optimality. The same four conditions reappear as the framework for constrained optimization in machine learning. (Complementary slackness here is the same condition met for LP duality in the previous chapter.)

## 3 Convergence theory

The speed at which iterative methods converge is controlled by two constants of the objective $f$ (assumed differentiable and convex).

- **Lipschitz gradient**: $f$ has an ***L*-Lipschitz gradient** if

  $$\|\nabla f(x) - \nabla f(y)\| \le L\|x - y\| \quad \text{for all } x, y.$$

  $L$ bounds how fast the gradient can change; for twice-differentiable $f$ it is an upper bound on the eigenvalues of the Hessian.
- **Strong convexity**: $f$ is **$\mu$-strongly convex** ($\mu > 0$) if $x \mapsto f(x) - \frac{\mu}{2}\|x\|^2$ is convex; equivalently the Hessian eigenvalues are bounded below by $\mu$. Strong convexity gives a unique minimizer and forces curvature in every direction.

The **condition number** is

$$\kappa = \frac{L}{\mu} \ge 1.$$

A small $\kappa$ (well-conditioned, near-spherical level sets) means fast convergence; a large $\kappa$ (ill-conditioned, elongated level sets) means slow convergence, because gradient steps zig-zag across the narrow valley.

**Convergence rates** (for gradient descent, Section 4):

- $L$-smooth convex: the objective gap decreases at rate $O\big(\frac{1}{k}\big)$ after $k$ iterations (**sublinear**).
- $L$-smooth and $\mu$-strongly convex: **linear** convergence,

  $$f(x_k) - f(x^*) \le \left(1 - \frac{1}{\kappa}\right)^k (f(x_0) - f(x^*)),$$

  so the number of iterations to reach a target accuracy scales with $\kappa$.

# 4 Gradient descent

**Gradient descent** minimizes a differentiable $f$ by repeatedly stepping in the direction of steepest decrease, $-\nabla f$:

$$x_{k+1} = x_k - \eta_k \nabla f(x_k),$$

where $\eta_k > 0$ is the **step size** (learning rate).

## 4.1 Step size selection

- $\frac{1}{L}$ **rule**: for an $L$-smooth function the constant step $\eta = \frac{1}{L}$ guarantees decrease at every iteration and the rates of Section 3.
- **Line search**: choose $\eta_k$ each step by (approximately) minimizing $f(x_k - \eta \nabla f(x_k))$ over $\eta$, e.g. by backtracking — start from a large $\eta$ and halve it until a sufficient-decrease condition holds.

A step that is too large can overshoot and diverge; one that is too small wastes iterations.

## 4.2 Convergence analysis

Under $L$-smoothness with $\eta = \frac{1}{L}$, gradient descent converges at the rates stated in Section 3: $O\left(\frac{1}{k}\right)$ for convex $f$ and linearly with factor $\left(1 - \frac{1}{\kappa}\right)$ when $f$ is additionally $\mu$-strongly convex. Empirically, plotting $f(x_k) - f(x^*)$ (or $\|\nabla f(x_k)\|$) against $k$ on a log scale shows a straight line for linear convergence; its slope steepens as $\kappa$ decreases.

# 5 Stochastic gradient descent

When the objective is a sum (or average) over $N$ data points,

$$f(x) = \frac{1}{N} \sum_{i=1}^{N} f_i(x),$$

computing the **full** gradient $\nabla f = \frac{1}{N} \sum_i \nabla f_i$ costs one pass over all $N$ points per step. **Stochastic gradient descent** (SGD) replaces it with the gradient of a single point, or a small **mini-batch** $B$:

$$x_{k+1} = x_k - \eta_k \cdot \frac{1}{|B|} \sum_{i \in B} \nabla f_i(x_k).$$

## 5.1 Unbiased estimator

If the batch index (or indices) is drawn uniformly at random, the stochastic gradient is an **unbiased estimator** of the full gradient:

$$\mathbb{E}[\nabla f_i(x)] = \frac{1}{N} \sum_{i=1}^{N} \nabla f_i(x) = \nabla f(x).$$

So on average SGD moves in the true descent direction, even though any single step is noisy.

## 5.2 Mini-batches and variance reduction

A larger mini-batch averages more samples, reducing the **variance** of the gradient estimate (variance falls like $\frac{1}{|B|}$) at proportionally higher cost per step. The noise prevents SGD from converging to a point with a constant step size; a **decreasing** step size, or explicit **variance-reduction** techniques (e.g. SVRG, SAG, which periodically use a full-gradient correction term), restore convergence to the exact minimizer.

### 5.3 Comparison with batch GD

Batch (full) gradient descent takes a precise but expensive step; SGD takes many cheap, noisy steps. Per unit of computation SGD usually makes far faster initial progress on large datasets, which is why it is the workhorse for large-scale learning, at the cost of noisier convergence near the optimum.

## 6 The Adam optimizer

**Adam** (Adaptive Moment Estimation) augments SGD with per-parameter step sizes derived from running estimates of the first two moments of the gradient. Writing $g_k$ for the stochastic gradient at step $k$:

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1-\beta_1) g_k \quad \text{(first moment)} \\ v_k &= \beta_2 v_{k-1} + (1-\beta_2) g_k^2 \quad \text{(second moment)} \end{aligned}$$

(the square is elementwise), with decay rates $\beta_1, \beta_2 \in [0, 1)$, typically $\beta_1 = 0.9$, $\beta_2 = 0.999$.

### 6.1 Bias correction

Because $m_0 = v_0 = 0$, the estimates are biased toward 0 early on. Adam corrects this:

$$\hat{m}_k = \frac{m_k}{1-\beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1-\beta_2^k}.$$

### 6.2 Per-parameter adaptation

The update divides the (bias-corrected) mean gradient by the square root of the (bias-corrected) mean squared gradient, elementwise:

$$x_{k+1} = x_k - \eta \cdot \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon},$$

with a small $\varepsilon$ for numerical stability. Each coordinate thus gets its own effective step size: directions with consistently large gradients are damped, while directions with small or noisy gradients are amplified, which helps on ill-conditioned objectives.

## 7 Proximal methods

### 7.1 The proximal operator

For a convex function $g$ and parameter $t > 0$, the **proximal operator** is

$$\operatorname{prox}_{tg}(v) = \arg\min_x \left( g(x) + \frac{1}{2t}\|x - v\|^2 \right).$$

It returns a point that trades off making $g$ small against staying near $v$.

### 7.2 Soft thresholding for L1

These methods target **composite** objectives $F(x) = f(x) + g(x)$ where $f$ is smooth (differentiable) and $g$ is convex but possibly **non-smooth** — the canonical case being the L1 penalty $g(x) = \lambda\|x\|_1$, used to induce sparsity.

The proximal operator of $g(x) = \lambda\|x\|_1$ is the **soft-thresholding** operator, applied coordinate-wise:

$$\big(\mathrm{prox}_{t\lambda\|\cdot\|_1}(v)\big)_j = \mathrm{sign}(v_j)\max\big(|v_j| - t\lambda, 0\big).$$

It shrinks each coordinate toward 0 by $t\lambda$ and sets to exactly 0 any coordinate with $|v_j| \le t\lambda$ — the mechanism by which L1 produces sparse solutions.

### 7.3 ISTA and FISTA

**ISTA** (Iterative Shrinkage-Thresholding Algorithm) alternates a gradient step on the smooth part with the proximal operator of the non-smooth part:

$$x_{k+1} = \mathrm{prox}_{tg}(x_k - t\nabla f(x_k)).$$

It converges at rate $O\big(\frac{1}{k}\big)$. **FISTA** (Fast ISTA) adds a momentum (extrapolation) step and improves the rate to $O\big(\frac{1}{k^2}\big)$ at essentially the same per-iteration cost.

Proximal methods are preferred over standard gradient descent precisely when the objective has a non-smooth term (such as the L1 penalty) whose proximal operator is cheap to evaluate: plain gradient descent does not apply where the gradient does not exist, but the proximal step handles the non-smooth part exactly.

### 7.4 ADMM

**If time permits** — for problems with structured or separable constraints, the **Alternating Direction Method of Multipliers** (ADMM) splits the problem into subproblems solved in turn and coordinated through a dual variable, allowing each piece (e.g. a smooth loss and a non-smooth regularizer) to be handled by its own proximal step.

## 8 Non-convex optimization

When $f$ is **non-convex**, the guarantees of the convex setting are lost.

- **Local minima**: points that are optimal within a neighbourhood but not globally. A descent method can converge to one and stop short of the global optimum.
- **Saddle points**: stationary points ($\nabla f = 0$) that are minima along some directions and maxima along others — the Hessian is indefinite (mixed-sign eigenvalues).

### 8.1 The high-dimensional landscape

In high dimension, a stationary point is a local minimum only if **every** Hessian eigenvalue is positive. As dimension grows it becomes increasingly unlikely that all eigenvalues share the same sign, so **saddle points vastly outnumber local minima**. Saddle points, not local minima, are the dominant obstacle to optimization in high-dimensional problems such as neural-network training.

### 8.2 Mitigation strategies

- **Subgradient methods**: at points where $f$ is non-differentiable, replace the gradient by a **subgradient** — any vector $g$ with $f(y) \ge f(x) + g^T(y - x)$ for all $y$ — and step along it. (Convergence is slow, $O\Big(\frac{1}{\sqrt{k}}\Big)$, and a decreasing step size is required.)
- **Random restarts**: run the method from several random initial points and keep the best result found, increasing the chance of escaping a poor local minimum.
- The gradient **noise** in SGD, and the per-parameter adaptation of Adam, also help iterates move away from saddle points rather than stall at them.